Construction Robot Digital Twin

Autonomous Navigation to a Wall-Mounted AprilTag

TAN TIAN

CIVIL 8025 Project 4

ROS2 + Gazebo + Nav2 + Local Web Dashboard

April 30, 2026

Four-wheel inspection robot, physics-based simulation, SLAM-ready map workflow, and browser-based operator interface.

github.com/skywalker0525/8025_pj4

Presentation Roadmap

What I built

- A ROS2 construction robot digital twin
- Gazebo world with collision constraints
- Nav2 mission from start point A to AprilTag target

What I will show

- Robot model and control interfaces
- Navigation, SLAM, telemetry, and safety
- Final video demo after the slides

Project Goal

Background

Construction sites are dynamic, cluttered, and safety-critical. Autonomous mobile robots can support inspection by navigating site-like areas, checking target locations, and giving operators live feedback.

Inspection task

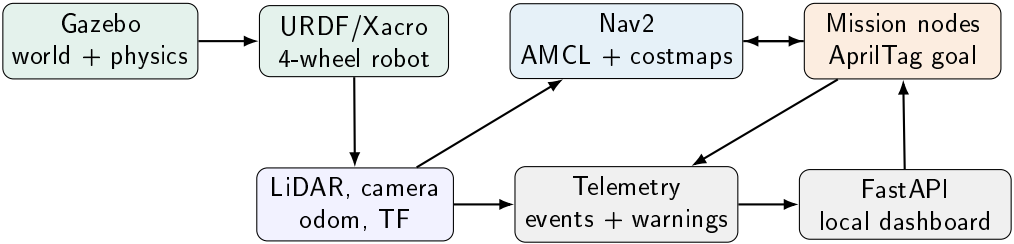
The robot starts at point **A**, autonomously navigates through a construction-like indoor environment, approaches a wall-mounted **AprilTag**, and reports the mission as complete.

- Mobile robot must obey physical and collision constraints.
- Operator can switch between automatic mission and manual control.
- Dashboard gives live feedback instead of relying only on RViz.

Task pipeline

1. Start from A.
2. Plan with Nav2.
3. Approach AprilTag.
4. Report `DONE`.

Digital Twin Architecture



The dashboard is an experience layer over ROS topics, actions, and services; it does not replace Gazebo or RViz engineering tools.

Robot and Environment Model

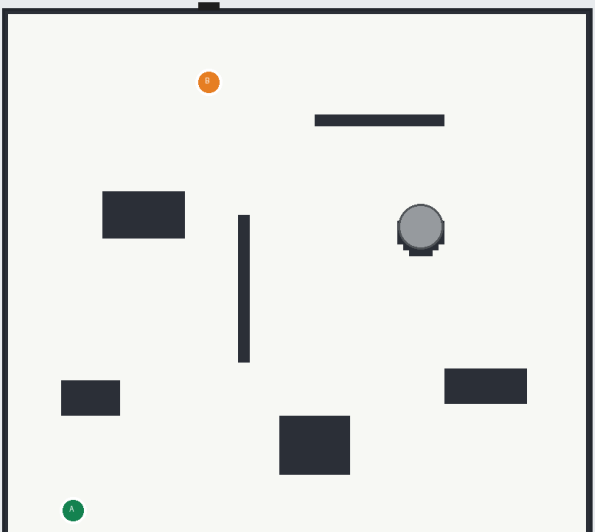
Robot

- Four-wheel differential/skid-steer base
- Rear left/right wheels driven by one Gazebo diff-drive plugin
- Passive front wheels for stable contact
- Pan/tilt camera gimbal with joint limits
- LiDAR, RGB camera, odometry, and TF

Construction scene

- Walls and floor with collision bodies
- Barriers, material stacks, tool chest, rebar cage
- Simple cylinder column for stable obstacle geometry
- Wall-mounted AprilTag target
- Static map plus SLAM workflow

Autonomous Mission



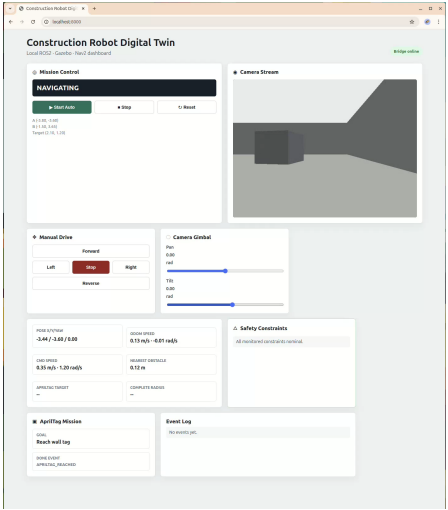
Mission sequence

1. Publish AMCL initial pose at A
2. Wait for `NAV2_READY`
3. Send B as `NavigateToPose`
4. Approach the AprilTag wall
5. Publish `APRILTAG_REACHED`

Map information

- Static map: `room_map.yaml`
- Resolution: 0.1 $m/cell$
- Size: 100×100 cells

Web Dashboard



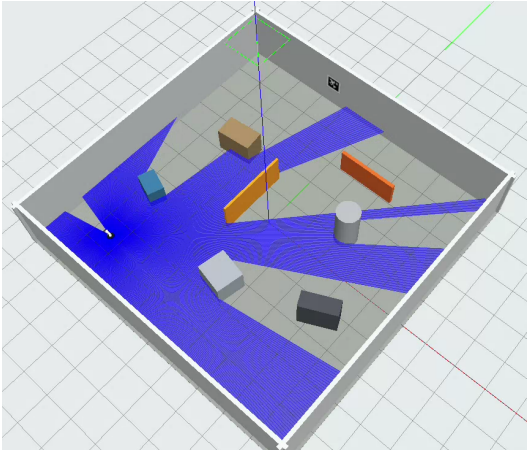
Operator controls

- Start Auto, Stop, Reset
- Manual forward, reverse, left, right
- Camera pan and tilt sliders
- Local camera stream endpoint

Live feedback

- Mission state and event log
- Pose, velocity, command velocity
- Nearest LiDAR obstacle range
- AprilTag target information
- Safety warning panel

SLAM and Map Workflow



Mapping mode

- Launch Gazebo with SLAM Toolbox
- Drive manually from the dashboard
- RViz shows the LiDAR-built `/map`
- Save the map for Nav2

Navigation mode

- Load `room_map.yaml`
- AMCL estimates `map -> odom`
- Diff-drive publishes `odom -> base_link`
- Nav2 plans through costmaps

Safety and Physical Constraints

Model-level constraints

- URDF collision geometry and inertia
- Wheel contact and friction parameters
- Gimbal pan/tilt joint limits
- Gazebo collision for walls and obstacles

Runtime constraints

- Nav2 obstacle and inflation layers
- Maximum linear and angular speeds
- Nearest obstacle distance warning
- Emergency `STOP` publishes zero velocity

Safety evidence is visible in code, costmaps, telemetry, and the dashboard.

Implementation Highlights

Simulation

`my_robot_sim`
Gazebo world, wall AprilTag, obstacles, robot spawn.

Navigation

`my_robot_navigation`
Nav2 parameters, map, waypoints, RViz, SLAM launch.

Mission + Web

`my_robot_mission`
Readiness checks, events, telemetry, manual control, markers.

- `START_AUTO` waits for Nav2 action server and `map -> base_link`.
- The mission reports failures explicitly using `NAV2_NOT_READY` events.
- The system can be run reproducibly with Docker and tmux.

Limitations and Future Work

Current limitations

- AprilTag is currently a visual target and known map goal.
- Dashboard shows telemetry, but RViz remains stronger for detailed costmap inspection.
- Real-world deployment would need calibration and sensor noise handling.

Next steps

- Add real AprilTag image detection overlay.
- Stream LiDAR/costmap view directly into the dashboard.
- Use tag pose estimation for final visual servoing.

Conclusion

What the project demonstrates

- A complete construction robot digital twin with physics, sensors, navigation, and operator interface.
- Autonomous navigation from A to a wall-mounted AprilTag target.
- Evidence of collision modelling, costmap-based planning, telemetry, and safety constraints.
- A reproducible Docker/tmux workflow for demonstration and future development.

Next: recorded mission video.